

GIT GUIA PARA INICIANTES >_

Commits, branches e fluxo de trabalho — passo a passo, com exemplos

Prof. Gustavo Franz (Science/Biology)

Python Developer in Progress

Building Educational Solutions with Python

GitHub: github.com/GustaFranz

O que é Git?

Git é um **sistema de controle de versão**. Ele registra o histórico do seu projeto, permite trabalhar em equipe, experimentar sem medo e voltar atrás quando necessário.

- ✓ Salva o histórico do projeto (commits).
- ✓ Permite trabalhar em várias linhas (branches).
- ✓ Facilita colaboração e revisão de código.
- ✓ Protege contra perda de código.
- ✓ Da liberdade para testar sem medo de errar.

O fluxo básico do Git

Arquivo alterado -> (git add) -> **Stage** -> (git commit) -> **Repositorio local** -> (git push) -> **GitHub (remoto)**.

- ✓ Você altera arquivos no projeto.
- ✓ **git add** coloca no stage (você escolhe o que será salvo).
- ✓ **git commit** salva no histórico.
- ✓ **git push** envia para o GitHub.

1. Stage (área de preparação)

O stage é uma área intermediária entre suas alterações e o commit. Você escolhe exatamente o que entra no próximo commit.

Comando	O que faz	Exemplo
<code>git add arquivo.py</code>	Coloca um arquivo específico no stage.	<code>git add app.py</code>
<code>git add .</code>	Coloca todos os arquivos no stage.	<code>git add .</code>
<code>git status</code>	Mostra o que mudou e o que está no stage.	<code>git status</code>
<code>git restore --staged x</code>	Tira do stage sem apagar as alterações.	<code>git restore --staged app.py</code>

Dica

Sempre use `git status` antes de commitar. Ele mostra exatamente o que esta pronto para ser salvo.

2. Commits (salvando alteracoes)

O commit e um 'ponto de salvamento' no historico. Cada commit deve ter uma mensagem clara sobre o que foi feito.

Comando	O que faz	Exemplo
<code>git commit -m "msg"</code>	Salva as mudancas do stage no historico.	<code>git commit -m "cria login"</code>
<code>git log</code>	Mostra o historico de commits.	<code>git log</code>
<code>git log --oneline</code>	Mostra o historico resumido (1 linha).	<code>git log --oneline</code>
<code>git diff</code>	Mostra o que foi alterado e ainda nao adicionado.	<code>git diff</code>

Exemplo de historico (git log --oneline)

```
1ab32cd  cria sistema de login
f4d9ab6  corrige botao de acesso
98f736e  ajusta layout da tela inicial
```

3. Branches (linhas de trabalho)

Branch e uma linha paralela de desenvolvimento. Permite criar funcionalidades sem mexer na branch principal (geralmente main).

Comando	O que faz	Exemplo
<code>git branch</code>	Lista todas as branches.	<code>git branch</code>
<code>git branch nome</code>	Cria uma nova branch.	<code>git branch tela-login</code>
<code>git checkout nome</code>	Muda para outra branch.	<code>git checkout tela-login</code>
<code>git checkout -b nome</code>	Cria e ja muda para a nova branch.	<code>git checkout -b pagamento</code>
<code>git switch nome</code>	Muda de branch (alternativa ao checkout).	<code>git switch main</code>
<code>git switch -c nome</code>	Cria e ja muda (alternativa).	<code>git switch -c nova-feature</code>

4. Mesclando (merge)

Quando a funcionalidade da branch estiver pronta, juntamos (merge) na branch principal.

Comando	O que faz	Exemplo
<code>git checkout main</code>	Vai para a branch principal.	<code>git checkout main</code>
<code>git merge nome</code>	Mescla a branch informada na atual.	<code>git merge tela-login</code>
<code>git push</code>	Envia as mudancas para o GitHub.	<code>git push</code>

Dica

Sempre teste a branch antes do merge. Depois, voce pode excluir a branch que nao precisa mais.

5. Envio e atualizacao (GitHub)

Comando	O que faz	Exemplo
<code>git push -u origin nome</code>	Envia a branch pela primeira vez.	<code>git push -u origin feature</code>
<code>git push</code>	Envia as mudancas da branch atual.	<code>git push</code>
<code>git pull</code>	Baixa e aplica as alteracoes do GitHub.	<code>git pull</code>
<code>git fetch</code>	Baixa as alteracoes sem aplicar (so informa).	<code>git fetch</code>

Dica

Sempre de git pull antes de comecar a trabalhar, para evitar conflitos.

6. Fluxo de trabalho recomendado

Siga esta ordem sempre que for criar uma nova funcionalidade:

Ordem dos comandos

```
git pull # atualiza a branch principal
git checkout -b nova-funcionalidade # cria e entra na branch
# ... faca suas alteracoes no codigo ...
git add . # coloca tudo no stage
git commit -m "mensagem clara" # salva no historico
git push -u origin nova-funcionalidade # envia para o GitHub
# abra um Pull Request, faca o merge na main e exclua a branch
```

Conceitos importantes

Termo	Significado
Repositorio	Projeto onde o codigo e gerenciado.
Commit	Salvamento de um ponto da alteracao.
Branch	Linha paralela de desenvolvimento.
Merge	Juncao de uma branch em outra.
Stage	Area de preparacao do commit.
Push	Enviar alteracoes para o GitHub.
Pull	Baixar alteracoes do GitHub.
Remote	Repositorio na nuvem (ex.: GitHub).

Comandos de emergencia (use com cuidado)

Comando	O que faz	Exemplo
<code>git restore arquivo</code>	Descarta alteracoes (volta ao ultimo commit).	<code>git restore app.py</code>

Comando	O que faz	Exemplo
<code>git reset --soft HEAD~1</code>	Desfaz o ultimo commit, mantendo as alteracoes.	<code>git reset --soft HEAD~1</code>
<code>git reset --hard HEAD~1</code>	Desfaz o commit e APAGA as alteracoes (perigoso).	<code>git reset --hard HEAD~1</code>

Resumo rapido + boas praticas

- ✓ Comandos do dia a dia: status, add ., commit -m, push, pull.
- ✓ Faça commits pequenos e com mensagens claras e objetivas.
- ✓ Use uma branch para cada funcionalidade ou correcao.
- ✓ Sempre teste antes do merge e mantenha a main estavel.
- ✓ Pratique e erre sem medo: com o tempo, vira natural.

Prof. Gustavo Franz (Science/Biology)

Python Developer in Progress

Building Educational Solutions with Python

Professor de Ciencias e Biologia desde 2013 — atualmente estudando Programacao.

Eu crio estes resumos para organizar os assuntos e estudar de forma clara e objetiva. Estou compartilhando porque eles tambem podem ajudar outros desenvolvedores que estao começando. Bons estudos — vamos aprender juntos!

GitHub: github.com/GustaFranz

Exercicios Python: github.com/GustaFranz/python_exercises

Para quem quiser ver a resolucao dos meus exercicios em Python.

EasyAnsi: github.com/GustaFranz/easyansi

Para quem quiser codificar personalizando com cores e estilos de forma mais facil, pratica e intuitiva.